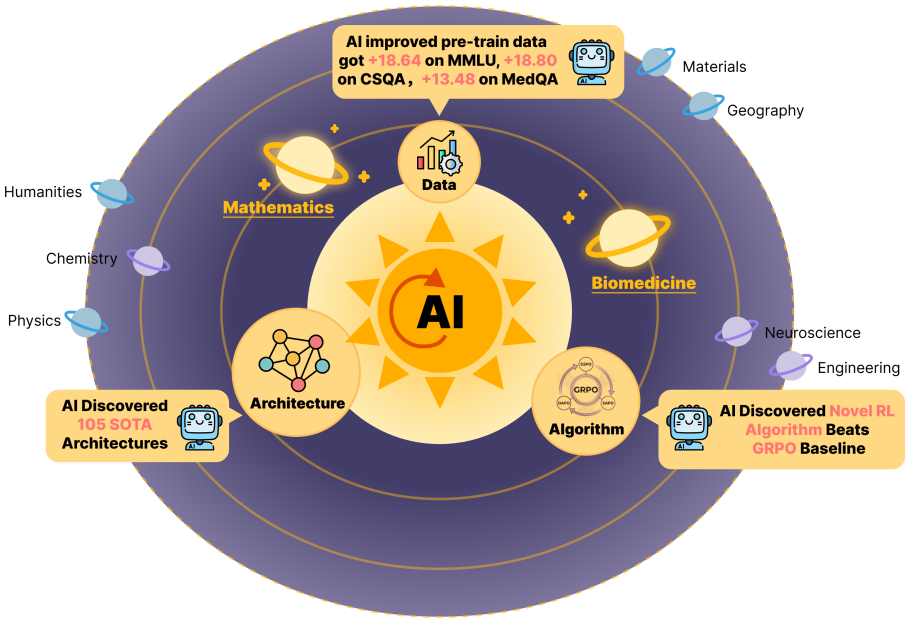
ASI-Evolve: AI Accelerates AI

Weixian Xu[‡1,2,3] Tiantian Mi[*1,2,3] Yixiu Liu[*1,2,3] Yang Nan[*2,3] Zhimeng Zhou[*2] Lyumanshan Ye[1,2,3]
Lin Zhang[1,2,3] Yu Qiao[2] Pengfei Liu[†1,2,3]

[1]SJTU [2]SII [3]GAIR

Abstract

Can AI accelerate the development of AI itself? While recent agentic systems have shown strong performance on well-scoped tasks with rapid feedback, it remains unclear whether they can tackle the costly, long-horizon, and weakly supervised research loops that drive real AI progress. We present ASI-EVOLVE, an agentic framework for AI-for-AI research that closes this loop through a learn–design–experiment–analyze cycle. ASI-EVOLVE augments standard evolutionary agents with two key components: a cognition base that injects accumulated human priors into each round of exploration, and a dedicated analyzer that distills complex experimental outcomes into reusable insights for future iterations. To our knowledge, ASI-EVOLVE is the first unified framework to demonstrate AI-driven discovery across three central components of AI development: **data, architectures, and learning algorithms**. In neural architecture design, it discovered **105 SOTA linear attention architectures**, with the best discovered model surpassing DeltaNet by +0.97 points (nearly 3× the gain of recent human-designed improvements). In pretraining data curation, the evolved pipeline improves average benchmark performance by +3.96 points, with gains **exceeding 18 points on MMLU**. In reinforcement learning algorithm design, discovered algorithms outperform GRPO by up to **+12.5 points on AMC32, +11.67 points on AIME24, and +5.04 points on OlympiadBench**. We further provide initial evidence that this AI-for-AI paradigm can transfer beyond the AI stack through experiments in **mathematics** and biomedicine. Together, these results suggest that ASI-EVOLVE represents a promising step toward enabling AI to accelerate AI across the foundational stages of development, offering early evidence for the feasibility of closed-loop AI research. The ASI-EVOLVE is fully open-sourced at https://github.com/GAIR-NLP/ASI-Evolve.



‡ Leading author.
* Core contributors.
† Corresponding author.

1 Introduction

Artificial intelligence (AI) advances through many interacting factors; **data, model architectures, and learning algorithms** are three central research components. Progress in each of these directions depends on repeated cycles of hypothesis generation, implementation, experimentation, and analysis (Ghareeb et al., 2025). In practice, however, these cycles are constrained by multidimensional human bottlenecks (Zhang et al., 2025): the hypothesis space humans can explore in parallel is severely limited (Liu et al., 2025a), experimental workflows demand substantial manual effort and frequent intervention (Feng et al., 2025), and the accumulation of insights across iterations often depends on individual experience and intuition, making knowledge difficult to systematically preserve and transfer (Kosmyna et al., 2025). Together, these constraints fundamentally limit the pace and scale of progress in AI development, raising a central question: *can AI accelerate the development of AI itself?* (Wang et al., 2023)

Recent advances in AI capabilities have made this possibility increasingly plausible (Didolkar et al., 2024). The role of AI in scientific discovery has evolved rapidly (Wei et al., 2025): from specialized systems that solve discrete, well-defined problems such as AlphaFold (Jumper et al., 2021), GraphCast (Lam et al., 2023), and GNoME (Merchant et al., 2023), to LLM-based and agentic systems that support broader scientific workflows. Systems such as SciMaster (Chai et al., 2025) focus on scientific question answering with known answers; ML-Master (Liu et al., 2025b) and MLEvolve (Du et al., 2025) address bounded optimization problems under fixed evaluation criteria; and AI Scientist (Lu et al., 2024) automates the research publication pipeline rather than tackling open-ended frontier research. AlphaEvolve (Novikov et al., 2025) takes an important step toward autonomous scientific optimization by iteratively improving candidate solutions through coding agents. Yet the research loops that drive real AI progress remain substantially harder to automate: improving architectures, data pipelines, or training algorithms typically requires modifying large codebases, running costly experiments, interpreting multidimensional outcomes, and sustaining coherent exploration across many rounds. Existing frameworks have not yet demonstrated that AI can operate effectively in this regime in a unified way, nor that it can generate meaningful advances across the **three foundational pillars of AI development** rather than within a single narrowly scoped setting.

To address this gap, we present ASI-EVOLVE, an agentic framework for **AI-for-AI** research. The general scientific process follows a principled cycle: researchers collect extensive background literature, formulate informed hypotheses, execute experiments, and distill insights through systematic analysis (Ghareeb et al., 2025). Inspired by this workflow, ASI-EVOLVE closes the loop between prior knowledge, hypothesis generation, experimental execution, and iterative refinement through a *learn–design–experiment–analyze* cycle. Two components are central to this design. First, a structured **cognition base** grounds each round of exploration in accumulated human research literature from the outset, allowing the system to build on domain knowledge rather than search from scratch. Second, a dedicated **analyzer** translates complex multi-dimensional experimental outcomes into structured, actionable insights that are written back into the experience database for future iterations. Together, these components enable sustained improvement on long-horizon AI research tasks where feedback is expensive, indirect, noisy, and difficult to interpret, substantially improving both the speed and quality of the evolution process.

Using ASI-EVOLVE, we demonstrate that AI can accelerate multiple parts of its own development stack. **To our knowledge, this is the first unified demonstration of AI-driven discovery across three central components of AI development: data, architectures, and learning algorithms. (1) Model Architecture**: In neural architecture design, ASI-EVOLVE autonomously generated 1,350 candidates across 1,773 exploration rounds, discovering 105 architectures that surpass the human-designed DeltaNet (Yang et al., 2025); its top-performing model achieved a +0.97 point gain, nearly triple the improvement of recent manual SOTA advancements (Dao and Gu, 2024). **(2) Data Curation**: In pretraining data curation, evolved strategies produced cleaner training datasets, improving over the original data by 3.96 points on average benchmarks, with particularly strong gains on knowledge-intensive benchmarks such as MMLU (Hendrycks et al., 2021), where improvements exceeded 18 points. **(3) Training Algorithm**: In reinforcement learning algorithm design, the framework derived novel optimization mechanisms with principled mathematical innovations that outperform the competitive GRPO (Guo et al., 2025) baseline by up to +12.5 points on AMC32, +11.67 points on AIME24, and +5.04 points on OlympiadBench. Together, these results show that under the ASI-EVOLVE framework, AI can accelerate key parts of AI development—from model architecture design to data preparation to learning algorithm development—forming a coherent, end-to-end closed loop for AI self-improvement.

We further validate the effectiveness of ASI-EVOLVE through targeted comparisons and ablation studies. On the circle packing task used as a shared benchmark across evolutionary frameworks, ASI-EVOLVE finds SOTA-level results in as few as 17 rounds, substantially outpacing prior frameworks including OpenEvolve and GEPA. To examine whether AI-designed components provide utility beyond the AI/ML stack, we additionally apply ASI-EVOLVE to drug-target interaction prediction, a biomedical domain distinct from AI development, where the evolved architecture achieves a 6.94-point AUROC improvement in cold-start generalization scenarios. These

results provide initial evidence that the AI-for-AI paradigm enabled by ASI-EVOLVE can generalize beyond AI tasks to broader scientific applications.

2 Preliminary

To systematically position existing work relative to ASI-EVOLVE, we introduce **Scientific Task Length** (L_{task}) as an analytical framework characterizing the intrinsic challenge of autonomous scientific research tasks along three dimensions: (1) Execution Cost (C_{exec}), which measures the computational resources and engineering complexity required per trial, including the burden of modifying large interdependent codebases and the GPU hours consumed. (2) Search Space Complexity (S_{space}), which captures the complexity of the solution space the system must navigate, including the openness of the task objective, whether candidate solution boundaries are predefined, and the extent to which meaningful exploration directions must be discovered rather than given. (3) Feedback Complexity (D_{feedback}), which measures the difficulty of extracting actionable insights from experimental outcomes, reflecting how much the system must synthesize multi-dimensional signals such as loss dynamics, benchmark distributions, and efficiency traces, rather than simply responding to a scalar score. We characterize task complexity as $L_{\text{task}} = \langle C_{\text{exec}}, S_{\text{space}}, D_{\text{feedback}} \rangle$ and use this lens to survey existing work.

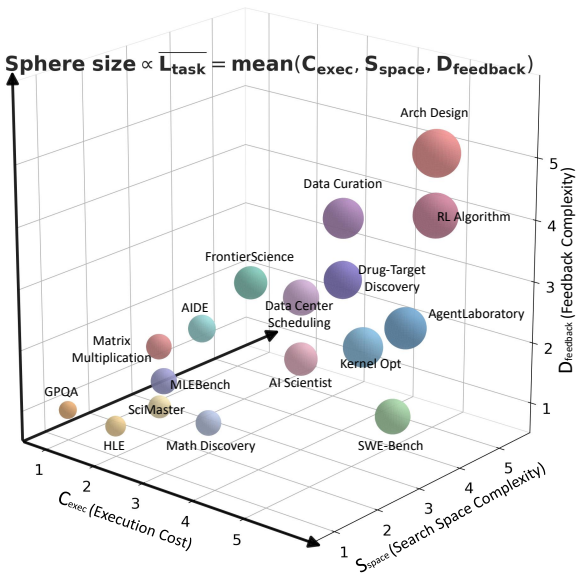


Figure 1: Representative scientific automation settings in the $L_{\text{task}} = \langle C_{\text{exec}}, S_{\text{space}}, D_{\text{feedback}} \rangle$ space.

Scientific question answering. This class of work involves little to no experimental execution; the task reduces to answering scientific questions against straightforward evaluation criteria, where correctness can be determined without interpreting complex feedback signals or iterative refinement. Idea generation systems (Wang et al., 2024a; Hu et al., 2024) and automated survey frameworks (Wang et al., 2024b) incur virtually zero C_{exec}. Scientific question answering benchmarks including GPQA (Rein et al., 2024), HLE (Phan et al., 2025), FrontierScience (Wang et al., 2026), and SciMaster (Chai et al., 2025) similarly operate under simple, univariate evaluation: no multi-dimensional experimental signals need to be synthesized, and no iterative experimental loop is required. All three dimensions of L_{task} remain low.

Structured task execution. Moving beyond pure reasoning, a number of systems introduce genuine experimental execution under clearly defined objectives. MLE-bench (Chan et al., 2024) and SWE-bench (Jimenez et al., 2024) require agents to optimize fixed targets in machine learning competitions or code repair; AIDE (Jiang et al., 2025) performs tree search over code space to optimize user-defined metrics. AI Scientist (Lu et al., 2024) and AgentLaboratory (Schmidgall et al., 2025) build more complete end-to-end pipelines, yet ultimately target structured, predefined tasks rather than open scientific discovery. Across this class, tasks follow established patterns with clear success criteria, and the goal is task completion rather than advancing scientific understanding. C_{exec} remains modest, and both S_{space} and D_{feedback} stay bounded: the exploration space is constrained by predefined objectives, and feedback signals, while present, do not require deep synthesis of multi-dimensional experimental outputs to guide meaningful scientific progress.

Lightweight scientific discovery. Further along the spectrum, evolutionary search frameworks achieve genuine open-ended discovery. AlphaEvolve (Novikov et al., 2025) improved Strassen's matrix multiplication algorithm for the first time in 56 years, advanced over 50 open mathematical problems, and optimized datacenter scheduling and the FlashAttention kernel. FunSearch (Romera-Paredes et al., 2023) discovered combinatorial optimization algorithms surpassing human-designed solutions. Mining Generalizable Activation Functions (Vitvitskyi et al., 2026) uncovered stronger generalizing activation functions through evolutionary search. OpenEvolve (Sharma, 2025), GEPA (Agrawal et al., 2026), ShinkaEvolve (Lange et al., 2025), AdaEvolve (Cemri et al., 2026), and SkyDiscover (Liu et al., 2026) further advance this paradigm in efficiency, diversity and adaptivity. In the L_{task} framework, S_{space} and C_{exec} are both elevated, as objectives are open-ended and iterative evaluation is required. Yet each trial remains small in scale, with modifications typically localized to a single function or short code segment. Feedback is thus immediate and direct, keeping D_{feedback} low.

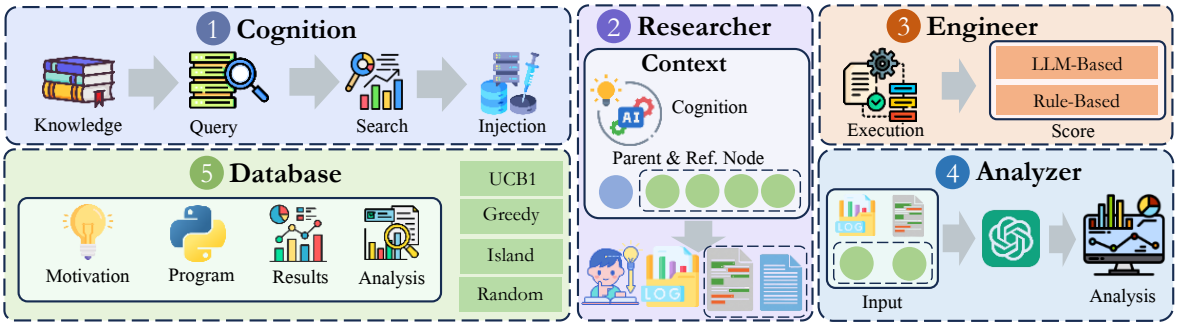


Figure 2: ASI-EVOLVE pipeline: in each round, the system samples context nodes from database, retrieves relevant cognition items via embedding search, generates a new candidate program, runs an experiment-specific evaluation script under timeouts, and summarizes results into an analysis report that is stored back into the database for future rounds.

Large-scale scientific exploration. At the upper end of the L_{task} spectrum lie tasks that push all three dimensions to substantially higher levels, occupying a region that existing systems have not yet addressed (Figure 1). Neural architecture design, pretraining data curation, and training algorithm design are foundational to AI progress, and represent three central components that ASI-EVOLVE targets. Validating a single candidate requires complete model training consuming tens to hundreds of GPU hours, often involving deep modifications to large interdependent codebases. The exploration space is broad and open-ended, spanning diverse design choices with no predefined boundaries. Experimental feedback spans multiple benchmarks, loss dynamics, and efficiency metrics, all of which must be jointly interpreted to guide the next iteration.

These properties impose unique demands on any system that attempts to automate such research. Each experimental trial is costly and opportunities for iteration are limited, meaning the system cannot afford to explore blindly. Prior domain knowledge must therefore be incorporated from the outset to steer exploration toward promising directions, motivating the **Cognition Base** in ASI-EVOLVE. At the same time, the richness of experimental feedback calls for dedicated interpretation: raw signals across benchmarks and training dynamics must be distilled into actionable insights before the next iteration can proceed, motivating the structured **Analyzer**. Together, these components reflect a key distinction between ASI-EVOLVE and existing evolutionary frameworks: while prior work evolves candidate solutions, ASI-EVOLVE evolves cognition itself. Accumulated experience and distilled insights are continuously stored and retrieved to inform future exploration, ensuring that the system grows not only in the quality of its solutions but in its capacity to reason about where to search next.

3 ASI-Evolve

ASI-EVOLVE is implemmented as an end-to-end experimental evolution pipeline. As illustrated in Figure 2, each iteration proceeds through four stages: (i) learn relevant knowledge and historical experience respectively from cognition storage and database, (iii) design the next candidate program, (iv) executes an experiment to obtain evaluation signals, and (v) analyzes outcomes into reusable, human-readable lessons. Below we describe the four corresponding modules in the actual system.

We view each evolution round t as searching over a program space $\mathcal{P}$ (code artifacts that implement a solution). The system maintains: (1) a database $\mathcal{D}$ of past nodes (motivation, code, results, analysis, score, and metadata), and (2) a cognition store C of task-relevant textual items indexed by embeddings. A new candidate program is generated conditioned on sampled context nodes $S_t \sim \text{Sample}(\mathcal{D})$ and retrieved cognition items $R_t = \text{Retrieve}(C;\, S_t)$,

$$p_t \sim P(p \mid S_t, R_t),$$

and is evaluated by an external, experiment-specific procedure that produces structured metrics with a primary scalar score. The resulting node is then appended to $\mathcal{D}$ for subsequent sampling.

3.1 Researcher

The Researcher generates the next candidate program given the task description, sampled context nodes, and retrieved cognition items. Each round it begins by sampling n nodes from the database, and then retrieving a small set of cognition items by semantic search over the sampled nodes' analyses or motivations to provide additional priors.

Conditioned on this context, the Researcher uses an LLM to produce a complete program together with a natural-language motivation, which are stored together as a new node for subsequent rounds. In addition to full-code generation, the system also supports an optional *diff-based* editing mode, where the model proposes localized modifications over a parent program; this incremental style is particularly helpful when evolving larger codebases over many rounds.

3.2 Engineer

The Engineer executes the candidate program in the actual experiment environment and produces the quantitative evaluation signal used for evolution. Given a generated program, it invokes a user-specified evaluation procedure that runs the experiment end-to-end and returns structured metrics, including a primary scalar score that serves as the fitness signal.

To better handle long-horizon tasks, the Engineer supports early rejection via configurable wall-clock limits and lightweight quick tests, improving efficiency by filtering flawed candidates before expensive runs. It can also optionally invoke an LLM-based judge to cover aspects of candidate quality that are difficult to capture with rule-based metrics alone, combining its score with the primary metric.

3.3 Analyzer

In our setting, the primary feedback used for selection is a scalar score produced by a task-specific evaluation, but the same run also yields rich auxiliary signals—multiple metrics, feature importances, training logs, and execution traces—that are useful for diagnosis yet too verbose to feed directly into subsequent rounds. This is particularly pronounced in the complex, large-scale tasks we target, where a single experiment may generate extensive logs spanning training dynamics, benchmark breakdowns, and efficiency traces. The Analyzer is designed to handle this asymmetry: it receives the current program together with the full experimental output—including raw logs and detailed metrics—and distills them into a compact, decision-oriented report. This full exposure allows the Analyzer to perform thorough causal analysis; the resulting report will be persisted in the database and used for retrieval in subsequent rounds, keeping context length manageable without sacrificing analytical depth.

3.4 Cognition

For long-horizon research tasks, exploration from scratch offers a larger hypothesis space but incurs substantial resource and time cost. We therefore introduce a **cognition base** that encodes **human prior knowledge**—task-relevant heuristics, known pitfalls, and design principles drawn from domain literature and prior runs—so that the system can be steered toward promising directions and iterate efficiently rather than rediscovering well-documented failure modes. In each round, after sampling context nodes from the database, the pipeline uses the sampled nodes' information as queries to *retrieve* a small set of similar cognition entries via embedding-based semantic search; these entries are then injected into the Researcher's context to guide hypothesis generation. Our experiments (see §5) show that equipping the loop with this cognition base significantly improves cold-start climb speed and iteration efficiency, without limiting long-term exploration capability.

3.5 Database

The database is the system's persistent memory: it stores the outcome of each evolution round and supplies the *sampled* nodes that form the Researcher's context. Whereas the cognition base provides fast-start prior knowledge, the historical nodes in the database convey **task-specific** information and become the dominant information source as evolution progresses, supporting sustained improvement beyond the initial climb. Each evolution step produces a *node* that stores: (i) the Researcher motivation, (ii) the generated program, (iii) structured results from the evaluation script, (iv) analysis report, and (v) auxiliary metadata such as runtime and success flag. For sampling, to support comparison and flexible deployment we encapsulate multiple policies behind a unified interface—UCB1, random, greedy, and MAP-Elites island algorithm. Our ablation studies (see §5) show that the choice of sampling algorithm strongly affects *sustained* improvement: unlike the cognition base, which primarily accelerates cold-start climb, different sampling strategies yield clearly distinct evolution trajectories and are therefore critical for tuning the system to a given task.

4 Main Tasks

We apply ASI-EVOLVE to three central components of AI development—model architecture design, training data preparation, and training algorithm design—covering key parts of the AI research pipeline from model structure to data to training. Each task shares a set of characteristics that make autonomous research particularly challenging: limited available prior knowledge specific to the task, long iteration cycles, substantial implementation complexity, and experimental feedback that is indirect, multi-dimensional, and difficult to interpret. These properties collectively

define a regime that poses significant challenges to automated scientific research, making them a demanding testbed for evaluating ASI-EVOLVE's capability to conduct autonomous AI research at scale.

4.1 Scenario 1: Model Architecture Design

Task Formulation Model architecture is a foundational component of AI systems, determining the capacity to model complex patterns, computational efficiency, and generalization. In this task, we focus on designing efficient sequence models through linear attention mechanisms. The quadratic complexity of standard Transformer attention ($O(N^2)$) has motivated extensive research into sub-quadratic alternatives—including DeltaNet (Yang et al., 2025), Gated DeltaNet (Yang et al., 2024), Mamba (Dao and Gu, 2024), and RWKV (Peng et al., 2023)—which achieve $O(N)$ complexity by decomposing attention computations or maintaining compressed memory states. Despite this progress, improving efficiency while preserving modeling capacity remains challenging, and the design space remains vast and under-explored. Using DeltaNet (Yang et al., 2025) as the baseline, the task requires the AI system to design novel attention layers with sub-quadratic complexity, employ chunk-wise computation patterns for efficient parallel training, and produce complete runnable implementations integrated into an existing large codebase.

Methodology We initialize the cognition repository with approximately 150 entries extracted from 100 papers on linear attention, state space models, and efficient transformers, providing the system with domain priors from the outset. The database uses a periodically refreshed candidate pool that retains the top 50 highest-scoring nodes; each round samples its root architecture from the top 10 and draws reference context from the broader top 50. Two characteristics of this task further motivate targeted engineering adaptations: each evaluation costs hours of GPU training, and the design space contains numerous hard constraints that are easy to violate. To improve runtime efficiency and constraint satisfaction, we introduce three mechanisms. A *static check agent* intercepts each generated design before training, verifying complexity bounds, chunk-wise structure, and causal mask correctness. A *debug agent* handles runtime implementation errors by inspecting error traces and attempting targeted fixes. A *novelty check* filters duplicate proposals via motivation similarity, encouraging genuine exploration.

We adopt a multi-stage evaluation strategy to balance exploration efficiency with result reliability. In the *exploration phase*, small models ($\sim$20M parameters: 8 layers, hidden dimension 256) are trained for 2000 steps on 1B tokens and evaluated on 10 core benchmarks with 500 samples each. Candidates are scored via a composite fitness that combines quantitative metrics from loss and benchmark scores after sigmoid normalization with LLM-as-a-Judge qualitative scores for code complexity, efficiency, and innovativeness; only architectures exceeding the baseline on both dimensions advance. In the *verification phase*, promising candidates are scaled to $\sim$340M parameters (24 layers, hidden dimension 1024) and trained on 1B tokens to verify that their gains persist under scaling. We also conduct additional validity checks, including causality tests to ensure that attention masks correctly prevent future information leakage. The top architectures then undergo *large-scale validation* at $\sim$1.3B parameters (24 layers, hidden dimension 2048), trained on 100B tokens, with evaluation expanded to 16 benchmarks including 6 held-out OOD test sets covering mathematics, code understanding, and multilingual tasks.

Results Over 1773 exploration rounds, **105** architectures surpassed the DeltaNet baseline in the verification phase. We selected 5 representative architectures spanning diverse design philosophies for large-scale validation. Table 1 presents their full performance across development and generalization benchmarks. On development benchmarks, these architectures achieve up to 57.28% average accuracy compared to DeltaNet's 55.76%; on generalization benchmarks, they reach up to 45.40% versus DeltaNet's 44.74%, confirming that gains transfer beyond the training distribution. Our best model achieves nearly **3**$\times$ the gain of the current human-designed SOTA (Mamba2's +0.34 points over DeltaNet (Dao and Gu, 2024)). This demonstrates that AI-driven evolution can discover architectures significantly outperforming human expert designs even in this high-saturation regime.

Analysis Analysis of the top 5 architectures reveals a consistent theme: moving beyond fixed allocation schemes toward adaptive, multi-scale routing that dynamically adjusts computational budget based on input content. **PathGateFusionNet** introduces hierarchical routing where a first-stage gate allocates budget between local and contextual processing, and a second stage distributes the contextual budget across short-range, long-range, and delta-rule update paths. **ContentSharpRouter** implements content-aware routing with learnable temperature parameters that prevent premature commitment to single pathways. **FusionGatedFIRNet** replaces softmax routing with independent sigmoid gates, allowing simultaneous activation of local and global paths alongside per-head retention parameters for the delta-rule memory path. **HierGateNet** employs two-stage gating with dynamic learnable floor values ensuring critical paths—especially the delta-path for long-range reasoning—never fully collapse. **AdaMultiPathGateNet** achieves token-level control via a unified BalancedSparseGate combining global, per-head, and per-token logits with entropy penalties preventing mode collapse. These architectures collectively demonstrate that principled adaptive routing, rather than fixed structural choices, is the key lever for improving upon the DeltaNet baseline.

Benchmarks	Human Discovered			AI Discovered				
	DeltaNet	Gated-DeltaNet	Mamba2	PG	C	FG	H	AM
Development								
Wiki ppl↓	17.00	16.84	16.66	16.18	16.05	**15.77**	16.65	16.26
LMB ppl↓	13.63	13.31	13.33	12.62	13.45	**12.34**	13.06	13.75
LMB	45.47	46.26	46.24	**47.60**	46.13	47.53	46.56	45.04
PIQA	73.12	74.10	73.78	72.91	74.37	72.91	**74.37**	74.10
Hella	56.29	57.55	**58.58**	56.99	57.00	58.47	56.85	57.17
Wino	55.88	58.01	58.48	57.22	57.85	**60.14**	57.38	57.62
ARC-e	73.40	72.14	72.98	73.06	72.05	74.28	73.11	**74.28**
ARC-c	**40.61**	36.95	39.33	40.36	39.76	40.02	39.33	39.33
SIQA	40.74	41.71	41.81	42.37	41.81	**42.78**	42.07	42.07
BoolQ	60.58	53.98	60.52	62.45	62.51	62.11	**63.03**	56.27
Generalization								
RACE	34.45	33.78	32.15	35.22	34.55	**35.60**	35.22	35.02
BBQ	29.53	29.75	29.43	29.95	30.55	**31.46**	30.88	30.27
MetaBench	26.97	28.67	27.70	25.64	**29.55**	26.79	29.38	28.98
QA4MRE	**40.00**	35.00	39.17	39.17	39.17	38.33	38.33	38.33
SCIQ	89.80	90.30	90.30	89.60	89.50	89.20	**90.40**	89.20
SWAG	47.69	48.17	**48.88**	48.22	47.80	48.57	48.18	47.80
Averages								
Dev. Avg	55.76	55.09	56.47	56.62	56.44	**57.28**	56.59	55.74
Gen. Avg	44.74	44.28	44.61	44.63	45.19	44.99	**45.40**	44.93
Overall Avg	51.04	50.46	51.38	51.48	51.61	**52.01**	51.79	51.11

Table 1: Top block: 10 development benchmarks, used in our exploration stage; middle block: 6 generalization benchmarks for out-of-distribution testing. **Bold** indicates the best result and underline is the suboptimal one. Model abbreviations are as follows: PG = PathGate-FusionNet, C = Content-SharpRouter, FG = FusionGated-FIRNet, H = Hier-GateNet, and AM = AdaMulti-PathGateNet.

We conduct multi-faceted analyses to understand why the framework works and how fitness guides the evolutionary trajectory. To study this guidance mechanism, we track the fitness and performance curves throughout the search. As expected, sigmoid normalization progressively compresses rule-based score differences in later rounds, giving high-scoring nodes relatively uniform sampling opportunities rather than allowing benchmark leaders to dominate. Correspondingly, 78% of high-performing architectures discovered in the second half are improvements built on designs found before round 900, indicating that ASI-EVOLVE follows the intended fitness design: benchmark-driven exploration in the early stage, followed by broader subjective refinement in later stages. To assess the role of cognition, we further compare design provenance across all 1773 architectures and the 105 SOTA architectures. Across the full population, 51.7% derive from the cognition base, 38.2% from accumulated experience, and 10.1% are novel; among SOTA architectures, the share from experience rises to 44.8% while novelty drops to 6.6%. This shift suggests that domain priors effectively accelerate cold-start search, while useful patterns are progressively distilled from the system's own trials as evolution proceeds. One notable limitation concerns computational efficiency: because the system operates at the level of attention mechanism design rather than low-level kernel implementation, it cannot directly produce hardware-optimized CUDA kernels. Although LLM-as-a-Judge scores penalize computationally expensive designs, this qualitative signal cannot guarantee that discovered architectures will match the wall-clock efficiency of top human-engineered implementations after full optimization.

4.2 Scenario 2: Pretraining Data Curation

Task Formulation In this task, the Evolve system must design category-specific curation strategies that improve pretraining data quality. Strategy design is inherently difficult: the strategy space is vast and discrete, encompassing choices of which operations to apply, how to specify decision criteria, and which quality issues to prioritize, with no clear mapping from design choices to effectiveness. For each category, experts must examine data samples to identify issues, explore this combinatorial space to formulate candidate strategies, write detailed specifications, validate results, and iteratively refine, a process requiring significant effort per category. This challenge scales with corpus heterogeneity: modern pretraining corpora comprise hundreds of categories spanning domains, content types and quality levels, each demanding independent strategy design. The AI must automate this process by observing data to identify issues, generating candidate strategies, evaluating effectiveness through diagnostic feedback, and refining iteratively to discover effective approaches across all categories.

Methodology We apply the ASI-Evolve framework to the pretraining data curation task. The cognition repository is initialized by examining sampled data from each category, storing identified quality issues such as HTML artifacts, incomplete fragments, formatting inconsistencies, and domain-specific noise patterns. In each iteration, the Researcher retrieves relevant quality issues from the cognition repository and generates candidate curation strategies. The Engineer executes these strategies on 500 sampled documents, applying the specified operations to produce cleaned versions. The Analyzer evaluate 50 randomly selected (original, cleaned) pairs, scoring each on a 1-10 scale. The Analyzer also provides diagnostic feedback on coverage (which identified issues were addressed) and executability (instruction clarity and consistency). The Database maintains complete records of all designed strategies with their scores and diagnostic analyses. Newly discovered quality issues during evaluation are added back to the cognition repository. These feedback mechanisms guide strategy refinement in subsequent iterations.

	Human Discovered				AI Discovered	
Benchmark	**Fineweb-Edu**	**Ultra-Fineweb**	**DCLM**	**Nemotron-CC**	**Nemotron-CC$_{ASI}$**	**Nemotron-CC$_{ASI+}$**
BBH	3.01	7.42	24.16	**26.82**	26.69	26.16
ARC-E	73.39	73.96	75.13	74.94	77.55	**78.59**
ARC-C	43.45	43.77	45.02	43.52	48.41	**49.32**
MMLU	28.38	25.53	28.54	27.49	32.55	**46.13**
AGIEval	16.96	17.72	17.90	18.15	**18.30**	18.21
HellaSwag	64.33	65.32	**70.39**	65.32	64.36	62.21
TriviaQA	0.67	0.42	**42.85**	25.33	26.96	26.65
RACE	35.43	34.28	**36.08**	35.04	35.63	34.28
DROP	6.78	7.78	**24.31**	19.57	19.48	18.49
WinoGrande	61.02	60.93	**64.99**	57.96	59.89	58.09
PIQA	75.80	75.63	**77.93**	76.79	76.80	76.15
CSQA	19.54	19.90	20.16	20.31	20.61	**39.12**
SIQA	45.02	43.43	**47.51**	44.36	43.58	43.57
OpenBookQA	39.92	39.84	**43.36**	39.80	42.20	41.44
GPQA	24.51	23.04	25.67	24.37	23.93	**27.10**
MedQA	26.36	24.84	24.88	26.77	26.11	**40.25**
MedMCQA	25.80	24.92	28.15	28.86	30.28	**40.97**
PubMedQA	67.04	64.44	66.56	67.68	**68.32**	67.68
Avg	36.52	36.29	42.42	40.17	41.20	**44.13**

Table 2: Benchmark comparison of models trained on different pretraining corpora and curation strategies. All models are 3B parameters trained on 500B tokens. Columns are grouped into Human baselines and ASI-Evolve discovered datasets.

Results The system successfully designed effective strategies for all selected categories from Nemotron-CC(Rabeeh Karimi Mahabadi, 2025) spanning 672B tokens across academic content in mathematics, computer science, medicine, and other STEM fields, each at two quality levels. Applying the optimized strategies produces Nemotron-CC$_{ASI+}$ (504B tokens). Training 3B-parameter models from scratch on 500B tokens and evaluating across 18 benchmarks, Nemotron-CC$_{ASI+}$ achieves 44.13 average score, surpassing raw data by 3.96 points and established corpora including DCLM, FineWeb-Edu, and Ultra-FineWeb under identical training budgets. Gains are particularly pronounced on knowledge-intensive tasks: **MMLU +18.64 points, CSQA +18.80 points, MedQA +13.48 points**.

Analysis We analyze the design characteristics of discovered strategies. Across all categories, the system converges on cleaning-focused approaches without any prescriptive guidance on which operations to employ, consistently combining targeted noise removal (HTML artifacts, duplicates, PII), format normalization (whitespace, punctuation), and domain-aware preservation rules. This convergence demonstrates that systematic cleaning with domain-specific preservation suffices for substantial quality improvements. Beyond this shared foundation, effective

Method	Math500	AMC32	AIME25	AIME24	OlympiadBench
$\text{GRPO}_{\text{Human}}$	82.0	67.5	20.00	20.00	45.92
$\text{Algorithm1}_{\text{ASI-Evolve}}$	85.6	80.0	29.58	31.67	50.96
$\text{Algorithm2}_{\text{ASI-Evolve}}$	84.6	77.5	23.75	23.33	48.74
$\text{Algorithm3}_{\text{ASI-Evolve}}$	84.8	77.5	30.00	30.00	49.18
$\text{Algorithm4}_{\text{ASI-Evolve}}$	82.4	75.0	20.00	20.00	46.81
$\text{Algorithm5}_{\text{ASI-Evolve}}$	82.0	72.5	23.33	20.00	45.62

Table 3: Performance comparison of evolved RL algorithms and the GRPO baseline on mathematical reasoning benchmarks. All methods were trained on Qwen-3-14B-base within the SIIRL framework using the Skywork OR1 dataset and evaluated after 250 training steps.

strategies exhibit consistent design patterns: concrete criteria with measurable thresholds, targeted deletion of specific elements, and explicit preservation rules that prevent over-aggressive filtering. The 2.93-point gap between optimized and suboptimal strategies further illustrates the value of iterative refinement. Rather than relying on one-shot generation, the agentic evolution process accumulates diagnostic feedback across iterations, analyzing what worked, what gaps remain, and how to improve, enabling the system to navigate the vast strategy space and converge on high quality solutions.

4.3 Scenario 3: Reinforcement Learning Algorithm Design

Task Formulation In this phase, we tasked the Evolve system with designing a novel Reinforcement Learning (RL) algorithm for Large Language Model (LLM) training. Using Group Relative Policy Optimization (GRPO) as the baseline, the objective was to redesign the mechanism for advantage allocation across sequences and the subsequent gradient computation. To succeed, the system was required to comprehend the mathematical foundations of RL, interpret diverse training metrics, and distinguish between stochastic training instability and genuine algorithmic improvements.

Methodology We initialized the cognition repository with 10 high-quality papers published subsequent to GRPO, covering variance reduction techniques and KL-penalty modifications. These entries provided the system with a preliminary understanding of the current research frontier, constraining the search space toward plausible mathematical directions while avoiding theoretical dead ends. We employed a two-stage validation protocol to balance computational cost and evaluation reliability. In the exploration phase, candidate algorithms were trained on a 4B parameter model for 150 steps and evaluated on 6 mathematics benchmarks. Promising candidates were then scaled to a 14B parameter model for 300 steps in the verification phase, with the evaluation suite expanded to Abstract Reasoning, STEM, Finance, and Coding domains to test generalization. For fitness scoring, we simplified the function relative to the architecture search by removing sigmoid normalization, instead using a linear weighted sum of benchmark accuracy and LLM-as-a-Judge qualitative scores, focusing directly on raw performance gains and algorithmic coherence.

Results Over the course of 300 evolutionary rounds, the system trained and evaluated a diverse array of policy gradient modifications, yielding 10 algorithms that outperformed the GRPO baseline in the exploration phase. Upon scaling to the 14B parameter verification phase, 3 algorithms demonstrated statistically significant improvements across all tested domains. On mathematical benchmarks, the best evolved variants improve over GRPO by +12.5 points on AMC32 (67.5 $\rightarrow$ 80.0), +11.67 points on AIME24 (20.00 $\rightarrow$ 31.67), and +5.04 points on OlympiadBench (45.92 $\rightarrow$ 50.96), suggesting that the system can effectively optimize subtle mathematical trade-offs in loss function design.

Analysis We highlight two representative high-performing algorithms that exhibit distinct theoretical innovations. (1) *Algorithm A (Pairwise Asymmetric Optimization)* introduces a *comparative advantage estimation*: instead of using a group mean, the advantage for a response A is calculated by averaging the $\tanh$-normalized pairwise reward differences against all other group samples ($R_A - R_B$). It further employs an *asymmetric clipping mechanism* that dynamically adjusts the PPO clipping window $[\epsilon_{down}, \epsilon_{up}]$ based on the sign of the advantage, and implements *High-Impact Gradient Dropout*, stochastically masking gradients for the most influential tokens (highest probability $\times$ advantage) to prevent overfitting to specific keywords. (2) *Algorithm B (Budget-Constrained Dynamic Radius)* adopts percentile-based normalization for advantage calculation, defined as $(r - c)/s$. Its core innovation is the *Global Update Budget* (z_{cap}): the algorithm dynamically assigns each token a trusted update radius inversely proportional to the magnitude of its advantage, and strictly enforces the exponential bound $\exp(c) \times |A| \leq z_{cap}$, mathematically guaranteeing that the total policy update magnitude remains within a pre-defined budget and effectively stabilizing training on noisy data. These two algorithms illustrate that ASI-EVOLVE can perform

rigorous mathematical derivation and discover principled solutions to fundamental stability and variance challenges in RL training, paralleling innovations seen in human-designed algorithmic advances.

5 Empirical Analysis

Having demonstrated ASI-Evolve's effectiveness across four complex real-world domains in Section 4, we now turn to a systematic empirical analysis of the framework. We first evaluate ASI-Evolve's performance on the circle packing task—a task that has been adopted by multiple evolutionary frameworks and thus enables direct comparison—to assess how our system compares against existing approaches and to examine the impact of design choices such as base model and sampling algorithm. We then conduct controlled ablation studies on the same task to isolate the contribution of individual components. Finally, we further demonstrate that the solutions produced by ASI-Evolve can be genuinely applied beyond the AI/ML stack: the model architecture evolved by ASI-Evolve achieves strong results in a biomedical setting, showing that AI-optimized designs carry practical value in real-world domains.

5.1 Benchmarking ASI-Evolve on Circle Packing

Task. We use the circle packing task from AlphaEvolve as a controlled evaluation platform. The problem requires placing 26 circles within a 1×1 square to maximize the sum of their radii. This is a classic combinatorial optimization problem with low verification cost, yet it still demands non-trivial algorithm design and iterative refinement, making it a suitable proxy for comparing evolutionary frameworks under aligned conditions.

Key results at a glance. Table 4 summarizes results across representative evolutionary frameworks. Our ASI-Evolve reaches 2.63597 in as few as 17 steps—the fastest among all compared systems—and achieves a best score of 2.635983, comparable to the top results reported by other frameworks.

Framework	Model	Rounds	Best Score
AlphaEvolve (Novikov et al., 2025)	Gemini 2.0 Flash + Claude 3.7	—	2.6359
OpenEvolve (Sharma, 2025)	Gemini 2.0 Flash + Claude 3.7	460	2.6343
LoongFlow (Team, 2024)	DeepSeek-R1-250528	—	2.6360
SkyDiscover	GPT-5	89	2.6360
ASI-Evolve **(Ours)**	GPT-5-mini	**17**	**2.6360**

Table 4: Circle packing comparison across evolutionary frameworks (26 circles in a 1×1 unit square; higher sum of radii is better). "Rounds" denotes the number of evolution steps to reach the reported best score; "—" indicates the value was not reported.

5.2 Comparison Experiments

5.2.1 Framework Comparison

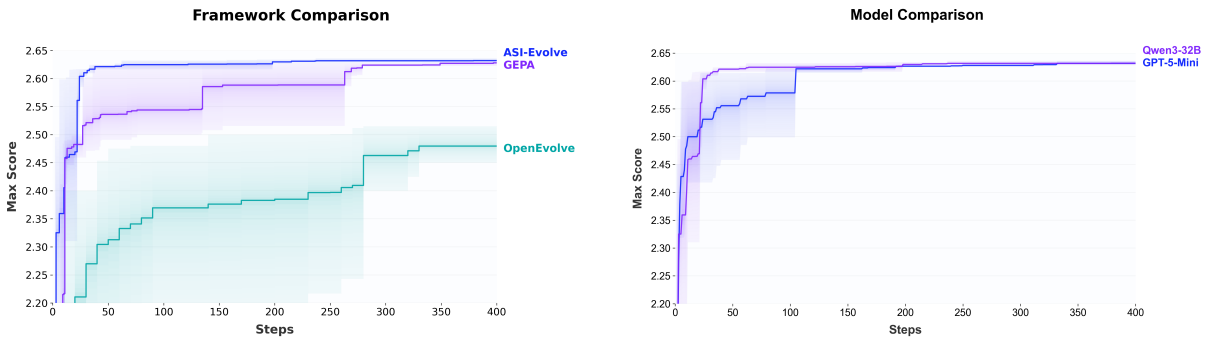
Using Qwen3-32B as the base model, we compare ASI-Evolve against two representative evolutionary frameworks—OpenEvolve and GEPA—under an aligned prompt setup. As shown in Figure 3(a), the three frameworks exhibit noticeably different evolution dynamics:

- **OpenEvolve** continues to evolve throughout the run, but exhibits high variance across independent runs and delivers only limited overall improvement, with scores plateauing well below the SOTA level.
- **GEPA** achieves competitive scores, converging to a range around 2.630. Its performance is substantially better than OpenEvolve, reflecting the benefit of structured evolutionary design.
- **ASI-Evolve** exits the cold-start phase with a noticeably higher score than both baselines, continues to improve steadily throughout the run, and is the only framework to reliably reach SOTA-level performance.

These findings align with our ablation results: the Analyzer and Cognition components provide structured feedback and domain priors that translate directly into faster convergence and higher final scores, advantages that persist across diverse evolutionary frameworks.

5.2.2 Model Comparison

We further run ASI-Evolve with GPT-5-mini and Qwen3-32B as the base model. As shown in Figure 3(b), the two runs converge to a similar range and exhibit highly consistent mid-to-late-stage improvement trends, indicating that the framework's evolution capability is not tied to a particular model family. At the same time, the early-stage cadence can differ across models: one model may enter the high-score regime earlier, while the other catches up via



(a) ASI-Evolve vs. GEPA vs. OpenEvolve. **(b)** Base-model comparison.

Figure 3: **Comparison experiments. (a)** Evolution curves (maximum score vs. step) for ASI-Evolve, GEPA, and OpenEvolve (all using Qwen3-32B). **(b)** Evolution curves for ASI-Evolve using GPT-5-mini and Qwen3-32B. Shaded regions indicate variability across repeated runs.

a distinct jump in the mid stage and ultimately converges to a comparable level. This further supports ASI-Evolve's compatibility and robustness across base models.

5.2.3 Algorithm Comparison

The database sampling algorithm determines how parent nodes are selected each round, directly shaping the balance between exploration and exploitation. **MAP-Elites** maintains a quality-diversity archive partitioned by behavioral features, actively preserving diverse niches to prevent premature convergence and encourage broad coverage of the solution space. **UCB1** treats each node as a bandit arm and selects based on an upper confidence bound that combines estimated value with an exploration bonus inversely proportional to visit count—rewarding high-scoring nodes while still visiting under-explored ones. **Random** sampling selects parent nodes uniformly at random from the database, without any preference for score or diversity.

As shown in Figure 4, the three sampling strategies exhibit distinct dynamics. **Random** achieves a higher initial score than MAP-Elites in early steps—because it places no diversity constraint on parent selection, the Researcher is free to immediately exploit the most informative nodes without being redirected toward under-explored regions. However, this early advantage erodes over time: without any mechanism to balance exploration and exploitation, Random sampling gradually slows its rate of improvement and falls behind both MAP-Elites and UCB1 in later stages.

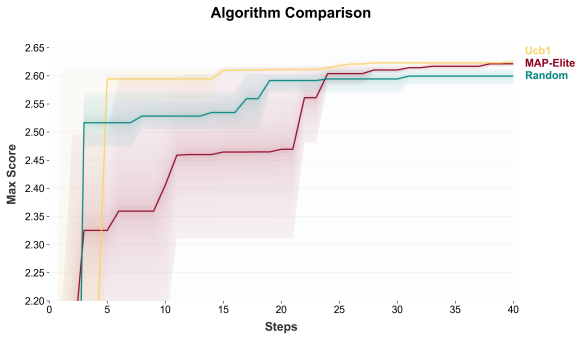


Figure 4: **Sampling algorithm comparison.** Evolution curves (maximum score vs. step) for ASI-Evolve with MAP-Elites, UCB1, and Random sampling on Qwen3-32B. Shaded regions show run-to-run variability.

UCB1 reaches high-score regions faster than MAP-Elites and exhibits lower variance across runs. This may appear counterintuitive—UCB1's exploitation bias could, in principle, cause it to converge prematurely on a narrow set of high-scoring parents and miss the broader diversity that MAP-Elites is designed to preserve. We attribute the reversed outcome to the role of cognition: with a well-initialized cognition repository and structured Analyzer feedback already providing directional guidance, the additional diversity enforced by MAP-Elites becomes less valuable, while UCB1's value-guided selection allows the system to rapidly concentrate on productive design patterns. These results indicate that, in the presence of cognition, the system can relax its dependence on diversity-preserving samplers and thereby achieve faster convergence. Notably, combining UCB1 with GPT-5-mini, the system discovered a circle-packing solution scoring 2.63597—matching the SOTA level—in just 17 steps. By contrast, MAP-Elites with the same GPT-5-mini base model required 79 steps to reach an equivalent score (2.63597), illustrating how exploitation-oriented sampling, when guided by strong cognition priors, can dramatically accelerate the discovery of high-quality solutions.

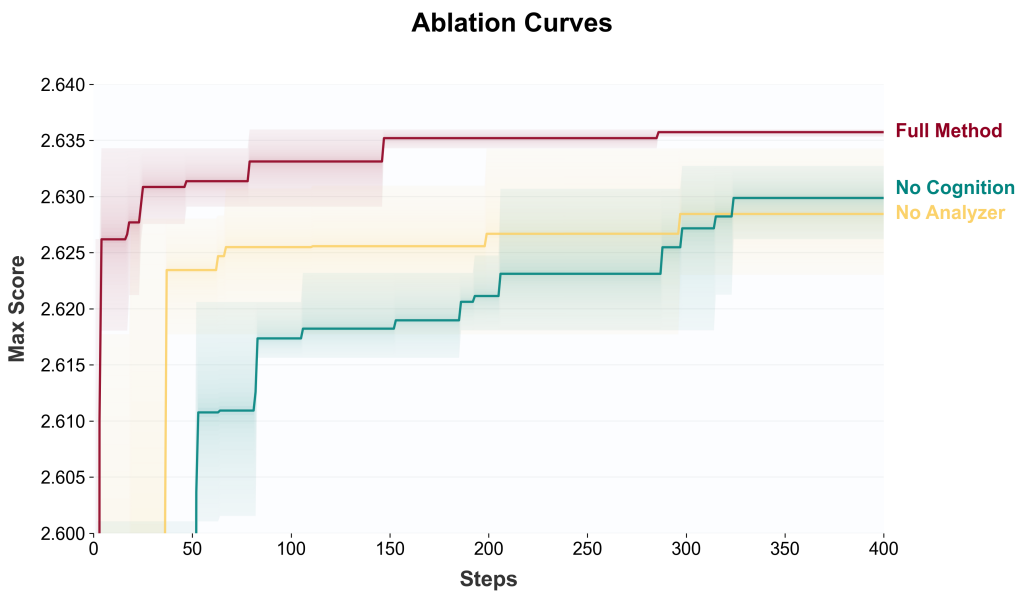


Figure 5: **Ablation on circle packing.** Evolution curves (maximum score vs. step) for the full method and ablated variants. Shaded regions indicate variability across repeated runs.

5.3 Ablation Study: Validating Component Effectiveness

5.3.1 Ablation design

We design the following controlled experiments to systematically evaluate key components of the ASI-Evolve framework:

1. **Full Method:** ASI-EVOLVE with Analyzer, Cognition repository, and the complete four-stage loop (Learn–Design–Experiment–Analyze).
2. **No Analyzer:** Remove the Analyzer module. After the Engineer runs experiments, raw evaluation scores and execution logs are stored directly in the Database as results for the next Researcher iteration.
3. **No Cognition:** Remove the Cognition repository. The Researcher receives no literature-derived prior knowledge; the system relies entirely on self-driven trial-and-error learning.

Considering the high variance inherent to evolutionary systems, we run each configuration three times independently, and analyze the overall improvement rate, convergence behavior, and cross-run gaps from evolution curves.

5.3.2 Impact of removing Analyzer

Figure 5 compares evolution curves with and without the Analyzer. We observe:

High initial scores attributable to Cognition. Even without the Analyzer, the No Analyzer variant begins from a relatively high score in the early phase. We attribute this to the Cognition repository: domain priors from literature guide the Researcher toward promising directions from the outset, regardless of whether structured feedback is available. This confirms that Cognition provides a meaningful cold-start advantage independently of Analyzer-driven feedback.

Long plateau with limited sustained improvement. Despite the high starting point, the No Analyzer variant subsequently enters a prolonged plateau where further iterations yield only marginal gains. The ability to continuously push toward higher ceilings becomes markedly weaker, and improvements become sporadic and less reproducible. Notably, some runs do eventually reach SOTA-level scores; we attribute this to two factors: first, even without a dedicated Analyzer, the system still receives a limited feedback signal through raw evaluation scores, providing some directional guidance; second, the circle packing task is relatively straightforward, so the performance gap between configurations is less pronounced than it would be on more complex tasks. Nevertheless, the absence of structured analysis consistently leads to slower and less reliable sustained evolution compared to the full method.

5.3.3 Impact of removing Cognition

Figure 5 also includes evolution curves with and without the Cognition repository. Key observations:

Discovered by	Model	BindingDB-Dev AUROC	BindingDB-Dev F1	BindingDB-Random AUROC	BindingDB-Random F1	Human AUROC	Human F1	BioSNAP AUROC	BioSNAP F1
Human	TransformerCPI	-	-	93.96	87.02	96.03	91.35	86.35	78.53
	PSICHIC	-	-	91.67	83.30	98.55	94.85	91.64	84.87
	ConPlex	-	-	93.59	83.39	97.04	90.38	88.40	72.86
	ColdStartCPI	-	-	89.27	85.27	98.29	93.57	93.39	86.84
	DrugBAN	94.15	86.89	94.89	87.96	98.61	95.40	89.43	82.69
AI	**ASI-Evolve**	**96.06**	**89.84**	**95.94**	**89.35**	**98.89**	**95.32**	**89.68**	**82.92**

Table 5: Performance comparison on drug-target interaction prediction across multiple benchmarks. We report AUROC (%) and F1 (%) scores for key evaluation settings. Bold indicates best performance.

Delayed exploration onset. The No Cognition variant exhibits a more pronounced cold-start cost: early improvements are slower and less stable, and the curve can remain in a relatively low-score region for a prolonged period. After sufficient effective experience is accumulated, the curve shows a noticeable jump and then gradually enters a higher-scoring, productive exploration regime. This matches the intended role of Cognition: it does not change the framework's core learning mechanism, but provides better priors to reduce unproductive exploration and shorten the "trial" phase.

Sustained evolution capability. Despite a slower start, the No Cognition variant still maintains effective evolution capability. Even without external priors, ASI-Evolve's core mechanisms can continuously learn from self-guided trial-and-error and gradually distill effective strategies. This also implies that in entirely novel domains where effective priors are unavailable, the framework remains usable, albeit requiring longer exploration.

5.4 Validating Real-World Applicability: Drug–Target Interaction Discovery

The experiments above confirm that ASI-Evolve delivers strong results on AI-for-AI tasks and that its components contribute meaningfully to performance. Yet a legitimate concern persists: even if AI can effectively optimize AI systems, whether the resulting solutions are genuinely useful when deployed in the real world. To address this directly, we present results on drug-target interaction (DTI) prediction, where the architecture evolved by ASI-Evolve is applied to a biomedical task to demonstrate that AI-optimized designs carry practical value beyond the AI/ML domain.

Task Formulation We apply ASI-Evolve to Drug-Target Interaction (DTI) prediction, a central problem in AI-driven drug discovery. Effective DTI models must simultaneously capture modality-specific representations of drug molecules and protein targets, as well as their complex interaction patterns. The architectural design space is large and discrete—spanning feature extraction, fusion mechanisms, and interaction modeling—with limited theoretical guidance. We use DrugBAN (Bai et al., 2022) as the seed architecture and aim to discover improved variants through automated architectural evolution.

Methodology The cognition repository is initialized from approximately 80 papers on graph neural networks, attention mechanisms, and DTI modeling, capturing known limitations such as over-reliance on shallow cross-attention and insufficient higher-order interaction modeling. In each round, the Researcher proposes and implements candidate architectural modifications (e.g., restructuring the drug-protein interaction module or introducing new cross-modal fusion strategies); the Engineer trains the modified DrugBAN on the BindingDB development set; and the Analyzer evaluates performance across benchmark splits and metrics (AUROC, AUPRC, F1, MCC), flagging failure modes such as sensitivity to protein length or chemical scaffold diversity. We evaluate on four datasets (BindingDB, Human, BioSNAP, C.elegans) under four generalization settings: **random split**, **unseen drug**, **unseen protein**, and **unseen drug and protein**. Fitness combines AUROC and AUPRC as primary objectives with F1 and MCC as secondary criteria. Over 100+ evolution rounds, 100+ candidate architectures were evaluated.

Results. Table 5 summarizes the performance of our best discovered architecture compared to DrugBAN baseline and six state-of-the-art baselines across multiple benchmark datasets and evaluation settings.

Our discovered architecture achieves consistent improvements over the DrugBAN baseline across most evaluation settings. On the BindingDB development set, we observe a substantial improvement of +1.91 AUROC points (0.9415 $\rightarrow$ 0.9606) and +2.95 F1 points (0.8689 $\rightarrow$ 0.8984), demonstrating effective in-distribution learning. Importantly, these improvements transfer to the test splits: on BindingDB-Random, our method achieves +1.05 AUROC points and +1.39 F1 points improvement. On the challenging Human benchmark, our architecture maintains strong performance with 0.9889 AUROC, while on BioSNAP it achieves modest but consistent gains (+0.25 AUROC, +0.23 F1).

Discovered by	Model	Unseen Drug		Unseen Protein		Unseen Drug and Protein	
		AUROC	F1	AUROC	F1	AUROC	F1
Human	DrugBAN	79.15	77.39	82.26	75.76	76.47	71.53
AI	**Ours**	**86.09**	**82.35**	**85.82**	**78.44**	**80.83**	**74.51**

Table 6: Cold-start performance comparison. We report AUROC (%) and F1 (%) scores. Models must predict interactions for unseen drugs, proteins, or both.

Generalization Analysis. We further analyze performance on cold-start scenarios where the model must generalize to completely unseen drugs or proteins (Table 6). These settings are particularly challenging as they require the model to extract transferable patterns from molecular structures rather than memorizing specific drug-target pairs.

The results reveal substantial generalization improvements: +6.94 AUROC points for unseen drugs, +3.56 points for unseen proteins, and +4.36 points in the doubly-cold-start setting. These improvements significantly exceed the in-distribution gains, suggesting that the evolved architecture has learned more robust and transferable representations of molecular interactions.

Analysis. The best discovered architecture (ban_sinkhorn_ds_marginal_topk_v6) introduces three key innovations over DrugBAN. (1) **Sinkhorn Attention**: replacing standard bilinear attention with optimal-transport-based Sinkhorn iterations enforces doubly-stochastic constraints, ensuring balanced attention allocation between drug and protein features and preventing attention collapse. (2) **Domain-Specific Marginalization**: specialized marginalization over molecular substructures (drugs) and protein domains aggregates interaction patterns across distinct semantic spaces, enabling more compositional modeling of binding mechanisms. (3) **Top-k Sparse Gating**: learnable top-k selection dynamically focuses on the most relevant interaction patterns, reducing noise from irrelevant molecular features. These choices align with domain knowledge—optimal transport has theoretical connections to binding affinity (Kumar et al., 2025), while compositional reasoning over substructures reflects established principles in medicinal chemistry.

Tracking the evolution process reveals that early iterations draw heavily on cognition from graph attention and molecular representation papers; as experiments accumulate, the system synthesizes cross-paper insights—most notably, the Sinkhorn mechanism emerged from combining optimal transport theory with bipartite matching concepts from computational biology. Consistent with this pattern, the fitness curve improves steadily overall, with notable jumps often appearing when cross-domain ideas are integrated successfully. Regarding common bottlenecks during evolution, the Analyzer repeatedly identifies failure modes such as representational collapse, overfitting to binding sites, and attention saturation, and the system progressively mitigates them through entropy regularization and orthogonality constraints.

Compared to expert-designed methods (TransformerCPI (Chen et al., 2020), PSICHIC (Koh et al., 2024), ConPlex (Singh et al., 2023), ColdStartCPI (Zhao et al., 2025)) that rely on pre-trained molecular encoders, protein language models, or specialized graph convolutions, our autonomously discovered architecture achieves competitive or superior performance across benchmarks. These results indicate that AI-driven architectural evolution can deliver real gains on challenging cross-domain tasks.

6 Conclusion

In this paper, we presented ASI-EVOLVE, an agentic evolution framework that enables AI to carry out end-to-end autonomous scientific research. Through controlled comparisons against existing evolutionary baselines and systematic ablation studies, we verified that the framework design is effective: equipped with a structured cognition base and a dedicated analyzer, the system achieves rapid cold-start and sustains continuous improvement, reliably reaching SOTA-level results.

We further explored whether AI can accelerate its own research pipeline across each stage of the scientific process. The closed *learn–design–experiment–analyze* loop enables efficient self-improvement, and we demonstrate breakthroughs across three central components of AI development—model architecture, training data, and training algorithms—each posing substantial challenges in terms of implementation complexity, iteration cost, and indirect feedback. Beyond the core AI pipeline, our drug-target interaction experiment demonstrates that model designs discovered through AI-driven research can be effectively deployed in real-world tasks, showing that AI-optimized solutions carry genuine scientific value.

Looking ahead, the scope of AI self-acceleration extends beyond individual models to the full AI development stack—architecture, data, algorithms, and infrastructure yet to be explored. As agentic systems take on more of the

implementation and iteration work, human scientists can shift from being the executors of solutions to the definers of problems—concentrating their expertise on the questions that matter most and leaving the expansive search through hypothesis spaces to AI. We expect this paradigm to drive not only the self-improvement of individual models, but the self-evolution of the entire AI field.

References

[1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. Gepa: Reflective prompt evolution can outperform reinforcement learning.

[2] Peizhen Bai, Filip Miljkovic, Bino John, and Haiping Lu. 2022. Interpretable bilinear attention network with domain adaptation improves drug-target prediction. *Nature Machine Intelligence*, 5:126–136.

[3] Mert Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei Zaharia, Alex Dimakis, and Ion Stoica. 2026. Adaevolve: Adaptive llm driven zeroth-order optimization.

[4] Jingyi Chai, Shuo Tang, Rui Ye, Yuwen Du, Xinyu Zhu, Mengcheng Zhou, Yanfeng Wang, Weinan E, Yuzhi Zhang, Linfeng Zhang, and Siheng Chen. 2025. Scimaster: Towards general-purpose scientific ai agents, part i. x-master as foundation: Can we lead on humanity's last exam?

[5] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Madry. 2024. Mle-bench: Evaluating machine learning agents on machine learning engineering.

[6] Lifan Chen, Xiaoqin Tan, Dingyan Wang, Feisheng Zhong, Xiaohong Liu, Tianbiao Yang, Xiaomin Luo, Kaixian Chen, Hualiang Jiang, and Mingyue Zheng. 2020. Transformercpi: improving compound-protein interaction prediction by sequence-based deep learning with self-attention mechanism and label reversal experiments. *Bioinformatics*, 36(16):4406–4414.

[7] Tri Dao and Albert Gu. 2024. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality.

[8] Aniket Didolkar, Anirudh Goyal, Nan Rosemary Ke, Siyuan Guo, Michal Valko, Timothy Lillicrap, Danilo Rezende, Yoshua Bengio, Michael Mozer, and Sanjeev Arora. 2024. Metacognitive capabilities of llms: An exploration in mathematical problem solving.

[9] Shangheng Du, Xiangchao Yan, Dengyang Jiang, Jiakang Yuan, Yusong Hu, Xin Li, Liang He, Bo Zhang, and Lei Bai. 2025. Automlgen: Navigating fine-grained optimization for coding agents. *arXiv preprint arXiv:2510.08511*.

[10] Haodong Feng, Lugang Ye, and Dixia Fan. 2025. Towards an ai fluid scientist: Llm-powered scientific discovery in experimental fluid mechanics.

[11] Ali Essam Ghareeb, Benjamin Chang, Ludovico Mitchener, Angela Yiu, Caralyn J. Szostkiewicz, Jon M. Laurent, Muhammed T. Razzak, Andrew D. White, Michaela M. Hinks, and Samuel G. Rodriques. 2025. Robin: A multi-agent system for automating scientific discovery.

[12] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K.

Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. 2025. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638.

[13] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring massive multitask language understanding.

[14] Xiang Hu, Hongyu Fu, Jinge Wang, Yifeng Wang, Zhikun Li, Renjun Xu, Yu Lu, Yaochu Jin, Lili Pan, and Zhenzhong Lan. 2024. Nova: An iterative planning and search approach to enhance novelty and diversity of llm generated ideas.

[15] Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. 2025. Aide: Ai-driven exploration in the space of code.

[16] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues?

[17] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. 2021. Highly accurate protein structure prediction with alphafold. *Nature*, 596(7873):583–589.

[18] Huan Yee Koh, Anh T. N. Nguyen, Shirui Pan, Lauren T. May, and Geoffrey I. Webb. 2024. Physicochemical graph neural network for learning protein-ligand interaction fingerprints from sequence data. *Nature Machine Intelligence*.

[19] Nataliya Kosmyna, Eugene Hauptmann, Ye Tong Yuan, Jessica Situ, Xian-Hao Liao, Ashly Vivian Beresnitzky, Iris Braunstein, and Pattie Maes. 2025. Your brain on chatgpt: Accumulation of cognitive debt when using an ai assistant for essay writing task.

[20] Ashutosh Kumar, Anubhav Raj, Rohit Gokhale, and Rohit Singh. 2025. Otmol: Robust molecular structure comparison via optimal transport. *Journal of Chemical Information and Modeling*.

[21] Remi Lam, Alvaro Sanchez-Gonzalez, Matthew Willson, Peter Wirnsberger, Meire Fortunato, Ferran Alet, Suman Ravuri, Timo Ewalds, Zach Eaton-Rosen, Weihua Hu, Alexander Merose, Stephan Hoyer, George Holland, Oriol Vinyals, Jacklynn Stott, Alexander Pritzel, Shakir Mohamed, and Peter Battaglia. 2023. Graphcast: Learning skillful medium-range global weather forecasting.

[22] Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. 2025. Shinkaevolve: Towards open-ended and sample-efficient program evolution.

[23] Shu Liu, Mert Cemri, Shubham Agarwal, Alexander Krentsel, Ashwin Naren, Qiuyang Mang, Zhifei Li, Akshat Gupta, Monishwaran Maheswaran, Audrey Cheng, Melissa Pan, Ethan Boneh, Kannan Ramchandran, Koushik Sen, Alexandros G. Dimakis, Matei Zaharia, and Ion Stoica. 2026. Skydiscover: A flexible framework for ai-driven scientific and algorithmic discovery.

[24] Yixiu Liu, Yang Nan, Weixian Xu, Xiangkun Hu, Lyumanshan Ye, Zhen Qin, and Pengfei Liu. 2025a. Alphago moment for model architecture discovery.

[25] Zexi Liu, Yuzhu Cai, Xinyu Zhu, Yujie Zheng, Runkun Chen, Ying Wen, Yanfeng Wang, Weinan E, and Siheng Chen. 2025b. Ml-master: Towards ai-for-ai via integration of exploration and reasoning.

[26] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*.

[27] Amil Merchant, Simon Batzner, Samuel S. Schoenholz, Muratahan Aykol, Anubhav Jain, and Ekin Dogus Cubuk. 2023. Scaling deep learning for materials discovery. *Nature*, 624(7990):80–85.

[28] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. Alphaevolve: A coding agent for scientific and algorithmic discovery.

[29] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, Xuzheng He, Haowen Hou, Jiaju Lin, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartlomiej Koptyra, Hayden Lau, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Guangyu Song, Xiangru Tang, Bolun Wang, Johan S. Wind, Stanislaw Wozniak, Ruichong Zhang, Zhenyuan Zhang, Qihang Zhao, Peng Zhou, Qinghua Zhou, Jian Zhu, and Rui-Jie Zhu. 2023. Rwkv: Reinventing rnns for the transformer era.

[30] Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, et al. 2025. Humanity's last exam. *arXiv preprint arXiv:2501.14249*.

[31] Shrimai Prabhumoye Mostofa Patwary Mohammad Shoeybi Bryan Catanzaro Rabeeh Karimi Mahabadi, Sanjeev Satheesh. 2025. Nemotron-cc-math: A 133 billion-token-scale high quality math pretraining dataset.

[32] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. 2024. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*.

[33] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2023. Mathematical discoveries from program search with large language models. *Nature*.

[34] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. 2025. Agent laboratory: Using llm agents as research assistants.

[35] Asankhaya Sharma. 2025. Openevolve: an open-source evolutionary coding agent.

[36] Rohit Singh, Sam Sledzieski, Lenore Cowen, and Bonnie Berger. 2023. Contrastive learning in protein language space predicts interactions between drugs and protein targets. *Proceedings of the National Academy of Sciences*, 120(24):e2220778120.

[37] Baidu Baige Team. 2024. Loongflow: Directed evolutionary search via a cognitive plan-execute-summarize paradigm.

[38] Alex Vitvitskyi, Michael Boratko, Matej Grcic, Razvan Pascanu, Deep Shah, and Petar Veličković. 2026. Mining generalizable activation functions.

[39] Hanchen Wang, Tianfan Fu, Yuanqi Du, Wenhao Gao, Kexin Huang, Ziming Liu, Payal Chandak, Chao Liu, Peter Van Katwyk, Andreea Deac, et al. 2023. Scientific discovery in the age of artificial intelligence. *Nature*, 620(7972):47–60.

[40] Miles Wang, Robi Lin, Kat Hu, Joy Jiao, Neil Chowdhury, Ethan Chang, and Tejal Patwardhan. 2026. Frontierscience: Evaluating ai's ability to perform expert-level scientific tasks.

[41] Qingyun Wang, Doug Downey, Heng Ji, and Tom Hope. 2024a. Scimon: Scientific inspiration machines optimized for novelty. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, page 279–299. Association for Computational Linguistics.

[42] Yidong Wang, Qi Guo, Wenjin Yao, Hongbo Zhang, Xin Zhang, Zhen Wu, Meishan Zhang, Xinyu Dai, Min Zhang, Qingsong Wen, Wei Ye, Shikun Zhang, and Yue Zhang. 2024b. Autosurvey: Large language models can automatically write surveys.

[43] Jiaqi Wei, Yuejin Yang, Xiang Zhang, Yuhan Chen, Xiang Zhuang, Zhangyang Gao, Dongzhan Zhou, Guangshuai Wang, Zhiqiang Gao, Juntai Cao, Zijie Qiu, Ming Hu, Chenglong Ma, Shixiang Tang, Junjun He, Chunfeng Song, Xuming He, Qiang Zhang, Chenyu You, Shuangjia Zheng, Ning Ding, Wanli Ouyang, Nanqing Dong, Yu Cheng, Siqi Sun, Lei Bai, and Bowen Zhou. 2025. From ai for science to agentic science: A survey on autonomous scientific discovery.

[44] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. 2024. Gated delta networks: Improving mamba2 with delta rule.

[45] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. 2025. Parallelizing linear transformers with the delta rule over sequence length.

[46] Sha Zhang, Suorong Yang, Tong Xie, Xiangyuan Xue, Zixuan Hu, Rui Li, Wenxi Qu, Zhenfei Yin, Tianfan Fu, Di Hu, Andres M Bran, Nian Ran, Bram Hoex, Wangmeng Zuo, Philippe Schwaller, Wanli Ouyang, Lei Bai, Yanyong Zhang, Lingyu Duan, Shixiang Tang, and Dongzhan Zhou. 2025. Position: Intelligent science laboratory requires the integration of cognitive and embodied ai.

[47] Qichang Zhao, Heng Zhao, Liang Guo, Ke Zheng, Yu Li, Qiang Ling, Jun Tang, Yang Li, and Jun Wang. 2025. Coldstartcpi: Induced-fit theory-guided dti predictive model with improved generalization performance. *Nature Communications*, 16:6436.

A Analysis Configuration Details

This appendix summarizes the concrete experiment settings used in the circle-packing analysis section. Unless otherwise noted, all results were reproduced from the archived runs under `Evolve/experiments/backup`.

Framework and model comparison (Section 5.2).

- **Compared settings:** aligned comparison with OpenEvolve-style prompting, and backbone comparison between GPT-5-mini and Qwen3-32B under the same system design.
- **Alignment protocol:** prompt style, full-file evolution mode, MAP-Elites/island database, and population-related settings were aligned to the OpenEvolve circle-packing setup.
- **GPT-5-mini setting:** temperature 0.7, top-$p = 0.95$, max tokens 32768.
- **Qwen3-32B setting:** temperature 0.6, top-$p = 0.95$, max tokens 65536, seed $= 42$, thinking enabled, top-$k = 20$, and min-$p = 0$.
- **Shared system setting:** max size $= 70$, `sample_n=3`, engineer timeout $= 300$s, 4 parallel workers, cognition retrieval top-$k = 5$, Judge disabled, and 3 runs per model.

Sampling algorithm comparison (Section 5.2).

- **Backbone model:** Qwen3-32B with the same decoding setting as above: temperature 0.6, top-$p = 0.95$, max tokens 65536, seed $= 42$, thinking enabled, top-$k = 20$, and min-$p = 0$.
- **Controlled factors:** prompt template, cognition setting, full-file evolution mode, engineer timeout, parallelism, and database size were held fixed.
- **Only changed factor:** database sampling algorithm. MAP-Elites uses the island configuration above, while UCB1 uses `algorithm=ucb1` with exploration constant $c = 1.414$.
- **Repeats:** 3 independent runs per sampling strategy.

Ablation study (Section 5.3).

- **Base model:** GPT-5-mini with temperature 0.7, top-$p = 0.95$, and max tokens 32768.
- **System loop:** Researcher–Engineer–Analyzer enabled by default; the "No Analyzer" setting disables Analyzer only, while "No Cognition" removes cognition usage but keeps the rest of the loop unchanged.
- **Researcher setting:** full-file generation (`diff_based_evolution=false`), max code length 100000, and up to 3 retries.
- **Engineer setting:** timeout $= 300$s, up to 2 retries, and 4 parallel workers.
- **Database setting:** max size $= 70$, sampling algorithm = island-based MAP-Elites, 5 islands, migration interval $= 10$, migration rate $= 0.1$, exploration ratio $= 0.2$, exploitation ratio $= 0.6$, feature dimensions = {`complexity`, `diversity`}, and 10 bins per feature.
- **Cognition setting:** retrieval top-$k = 5$, score threshold $= 0.4$, sentence-transformer embedding dimension $= 384$, and web search disabled.
- **Other shared settings:** `sample_n=3`, Judge disabled, and 3 independent runs per condition. All curves report the mean trend with variability across runs.

Cognition repository contents for circle packing. All experiments that use cognition share a common knowledge base initialized before the first evolution step. The 12 cognition items span three categories:

- **Geometric priors (4 items).**
  - *Hexagonal close packing:* theoretical density $\pi/(2\sqrt{3}) \approx 0.9069$ for infinite planes; unit-square boundary effects reduce achievable density; best patterns use hexagonal arrangements with careful corner and boundary handling.
  - *Edge and corner effects:* circles near boundaries are space-constrained; larger circles should be placed at corners and edges (corner radius up to $\frac{\sqrt{2}}{2}$ times the distance from the corner); small epsilon in overlap and bound checks avoids numerical tangency.
  - *Variable radii:* optimal solutions for $n = 26$ use circles of different sizes—larger circles at the center and corners, smaller ones filling gaps—with no uniform-radius assumption.
  - $n = 26$ *target knowledge:* the benchmark target is sum-of-radii ≈ 2.635 (AlphaEvolve); central hexagon plus outer layer arrangements work well; variable radii and corner optimization are the two most critical factors.
- **Optimization methodology (4 items).**
  - *SLSQP constrained optimization:* maximize sum of radii subject to explicit no-overlap ($d_{ij} \geq r_i + r_j + \varepsilon$) and in-bounds constraints via `scipy.optimize.minimize`; warm-start from high-scoring nodes (score ≥ 2.2).

  - *Multi-start strategy:* 3–5 different initial configurations from the best database nodes; optimize each independently and keep the best; tight tolerances with `maxiter` 500–1000.
  - *Differential evolution:* use `scipy.optimize.differential_evolution` for global refinement when local optimization plateaus; optionally refine radii only (lower-dimensional subspace), then polish with SLSQP.
  - *AlphaEvolve reference:* AlphaEvolve achieved 2.635 via constructor-plus-constrained-optimization; key insight is good geometric initialization followed by strict-constraint numerical refinement.
- **Engineering and troubleshooting guidelines (4 items).**
  - *Incremental refinement:* do not rewrite the full program; make targeted changes to optimizer settings, constraint formulation, multi-start count, or add a dedicated refinement stage.
  - *Code structure:* keep construction and optimization as separate stages; use a high-scoring node's code as base and apply diff-style edits.
  - *Numerical stability:* use a small epsilon (e.g. 10^{-8}) in all constraints to avoid numerical tangency; verify that no constraint is violated before reporting the sum of radii.
  - *Plateau-breaking checklist:* if stuck at ≈ 2.3–2.4, try (1) different initial pattern from a high-scoring node, (2) increase `maxiter` to 500–1000, (3) switch to explicit inequality constraints instead of penalty terms, (4) increase multi-start count to 3–5, (5) run differential evolution then SLSQP polishing.

Note on the framework comparison. OpenEvolve is commonly reported with a fast/slow model pair, where a second model can affect proposal quality and selection dynamics. In our aligned comparison, we instead instantiated both ASI-Evolve and OpenEvolve-style settings with the same backbone model in each run (either GPT-5-mini or Qwen3-32B), so that the comparison focuses on framework design rather than auxiliary-model choice. We acknowledge that this simplification may introduce bias relative to the best possible OpenEvolve configuration, since a different auxiliary model could potentially improve its performance. However, using the same model on both sides avoids an additional confound: results would otherwise depend heavily on how the second model is chosen, and a poorly chosen auxiliary model could distort the comparison even more. For GEPA, we retained all official settings except model-related parameters. We set the score of invalid evaluation outcomes to 0; otherwise the system would record invalid nodes with anomalously high scores and distort evolution. All other GEPA configuration and evaluation logic is unchanged from the official release.